U01155118

Lab 4 | Part I | Least Cost Path Routing

By Bhuwan Pandey

This lab aims to determine which nodes in a directed and weighted graph are guaranteed to appear on every least cost path from the source node to the destination. Because multiple shortest routes may exist, the challenge is to identify the nodes that all these minimum cost routes share. To handle large graphs efficiently, the solution uses Dijkstra's algorithm twice along with a shortest path DAG (Directed Acyclic Graph) and simple dynamic programming.

The program first runs Dijkstra's algorithm starting at node *1*, which gives the minimum distance from the source to every other node. Then it reverses all edges and runs Dijkstra again from the destination node *n*, which reveals the minimum distance from every node to the destination. With these two sets of distances, we can test whether an edge belongs to at least one shortest path. i.e. an edge *(u → v)* is valid if the distance to *u*, plus the edge cost, plus the remaining distance from *v to n*, matches the overall shortest distance from *1* to *n*. Only edges that satisfy this condition are included in a new graph structure, which becomes a DAG representing every shortest path in the network.

Once the DAG is constructed, the nodes are arranged in topological order. Using this order, the algorithm counts how many shortest paths reach each node from the source. Then, by processing the same nodes in reverse, it counts how many shortest paths go from each node to the destination. A node is considered mandatory if the number of paths passing through it matches the total number of shortest paths from start to end. In simpler terms, if every shortest path is forced to flow through that node, then the node must be included in the final result.

The core logic of the algorithm can be expressed briefly as:

```
// run Dijkstra from source (node 1)
long[] distFromSource = dijkstra(graph, 1);
// run Dijkstra on reversed graph from destination (node n)
long[] distToDestination = dijkstra(reversedGraph, n);
// shortest distance from 1 to n
long shortest = distFromSource[n];
// build shortest-path DAG
for each edge (u, v, cost):
   if (distFromSource[u] + cost + distToDestination[v] == shortest) {
      dag[u].add(v);
      indegree[v]++;
   }
// topological order of DAG
Queue<Integer> q = all nodes with indegree 0 AND reachable;
while (!q.isEmpty()):
   int u = q.poll();
   topoOrder.add(u);
   for (v in dag[u]):
      indegree[v]--;
      if (indegree[v] == 0) q.add(v);
// count number of paths from source
waysFromStart[1] = 1;
for (u in topoOrder):
   for (v in dag[u]):
      waysFromStart[v] += waysFromStart[u];
// count number of paths to destination
waysToEnd[n] = 1;
for (u in topoOrder reversed):
   for (v in dag[u]):
      waysToEnd[u] += waysToEnd[v];
// determine mandatory nodes
long totalPaths = waysFromStart[n];
```

```
for each node u:
   if (waysFromStart[u] > 0 &&
      waysToEnd[u] > 0 &&
      waysFromStart[u] * waysToEnd[u] == totalPaths) {
      add u to final result;
   }
```

This approach avoids enumerating individual paths and instead relies on distance consistency and simple arithmetic to determine mandatory nodes. Since the most expensive operations are the two Dijkstra runs, the overall time complexity is *O(m log n)*, which more or less fits the performance requirements for large graphs. The combination of two Dijkstra passes, the DAG filtering, and lightweight path counting results in a solution that is both efficient and straightforward made this solution to the problem, more approachable.